

Loading data

As discussed earlier, you can load multiple types of data using Pandas. I will create a simple csv file and load the data from that. For the purposes of this tutorial, I will use the examples of the current COVID-19 cases in different states across India.

States	Confirmed
Telangana	3496
Maharashtra	82968
Tamil Nadu	30152
Delhi	27654
Gujarat	19592

Figure 1: CSV data

In order to create a better visual for the data, I will use Jupyter Notebook (.ipynb) for this tutorial. However, you can create a simple .py file and run the program as well. In order to follow this tutorial using Jupyter notebook, do create a simple csv file as shown in Figure 1, and save it into the same folder as your .ipynb file.

Navigate to the folder of your csv file using the terminal and run Jupyter Notebook. The browser dashboard for Jupyter will open. Click *New* → *Python 3*. This will open up a fresh Python 3 notebook for you.

In the notebook, the first thing we need to do is import the Pandas library and load the csv file into a Pandas data frame. We do this using the `read_csv()` method in Pandas.

```
import pandas as pd

df = pd.read_csv('data.csv')
```

A data frame in Pandas is a two-dimensional data structure, used to visualise data in the rows and columns format.

Visualising a data frame

In order to visualise the data frame, you can simply type the name of the variable you used to store the data frame in (in this case, it's `df`) and run the code. That will give you a visual of the data as shown in Figure 2.

	States	Confirmed
0	Telangana	3496
1	Maharashtra	82968
2	Tamil Nadu	30152
3	Delhi	27654
4	Gujarat	19592

Figure 2: Pandas data frame

However, printing the entire data frame may not always be ideal, especially if the data set is huge. You can instead use the `head()` method in Pandas to visualise the first five rows of data from the data frame.

```
#Visualize the first 5 rows of data
frame
df.head()
```

You can also use the `shape` attribute in Pandas, which outputs a tuple that gives the total number of rows and columns in the data frame.

```
#Total no. of rows and columns
df.shape
```

In order to easily generate the basic statistical details pertaining to your data frame such as mean, standard deviation, etc, you can use the `describe()` function in Pandas that will visualise those details for you.

```
#Generate Basic statistical details
df.describe()
```

Figure 3 shows the statistical details

generated using the `describe()` method.

	Confirmed
count	5.000000
mean	32772.400000
std	29931.143126
min	3496.000000
25%	19592.000000
50%	27654.000000
75%	30152.000000
max	82968.000000

Figure 3: Output of the `describe()` method

Adding rows and columns

Let us add the confirmed cases of COVID-19 in Rajasthan to our data frame. In order to do this, we will have to add another row of data, for which we create a dictionary of the data that needs to be added and use the `append()` method in Pandas to add the data. Figure 4 shows the output after adding Rajasthan to the data frame.

```
#adding a row to data frame

row = {'States': 'Rajasthan',
       'Confirmed': 10331}
df = df.append(row, ignore_index=True)
df
```

	States	Confirmed
0	Telangana	3496
1	Maharashtra	82968
2	Tamil Nadu	30152
3	Delhi	27654
4	Gujarat	19592
5	Rajasthan	10331

Figure 4: Adding a row to data frame